

1AEL0256

Microcontroladores

Semana 03

Semestre 2026-2

Profesor: Kalun José Lau Gan

¿Qué hemos aprendido hasta el momento?

- Introducción al curso y repaso de conocimientos
- El microcontrolador PIC de Microchip
 - Familias
 - Herramientas de desarrollo: Proteus, MPLABX
- El microcontrolador PIC18F47Q10
 - Especificaciones técnicas iniciales
 - Cantidad de puertos
 - Cantidad de memoria
 - Especificaciones eléctricas
 - El primer ejemplo: compuerta NOT
 - Circuito en Proteus
 - Proyecto en el MPLABx
 - Código en ensamblador
 - Pruebas de simulación

Preguntas previas:

- Tengo errores al momento de realizar el primer ejemplo con el MPLAB X, me arroja "lexical error"

movlw 0x10 ↗
movlw 10H ↗
movlw 010H ✓

Muy probablemente error de formato de un número declarado en el código XC8 PIC Assembler.

Cuando el número hexadecimal inicia con una letra (A,B,C,D,E ó F), debes de añadirle un cero.

- ¿Cuál es la diferencia entre BRA y GOTO?
 - BRA saltos cortos (ocupa 2 bytes al usarlo)
 - GOTO salto largos (ocupa 4 bytes al usarlo)

Preguntas previas:

- He visto videos de youtube y algunos libros (el del pic16f84 supongo de José Angulo Usategui) donde hay algunas discrepancias en la representación de números en el Assembler
 - No hay discrepancias, un número en cualquiera de sus representaciones numéricas (hex, bin o dec) siempre es el mismo valor, muy posible que sea por la mala interpretación de las instrucciones empleadas.
 - movlb -> rango de 0 a 63
 - movlw -> rango de 0 a 255
 - lfsr -> rango de 0 a 16383
- ¿Es mejor emplear el ASM para cuando use periféricos como I2C, SPI, UART frente al XC8? Porque estuve haciendo unas pruebas de esos módulos en ASM y no me salía
 - No te debe de salir por no configurar correctamente los módulos, pero se recomienda utilizar lenguaje de alto nivel ya que luego de emplear esos módulos muy posiblemente necesites hacer operaciones matemáticas (escalamiento, filtrado, promediación, control de errores, etc) el cual de hacerlo en ASM te va a demorar demasiado tiempo en implementarlo, cosa que en XC8 alto nivel sería mas rápido de hacer ya que puedes hacer operaciones aritméticas complejas y de precisión con float.

Preguntas previas:

- No me compila el proyecto en el MPLABX y no sale donde esta el error en la ventana de output
 - Muy posiblemente se ha colocado caracteres restringidos en el nombre yó en la ruta del proyecto.
 - Ej. Semana_2,45 <- No debe de colocarse el punto en el nombre
 - Ej. Semana 2_3 <- De preferencia no dejar espacios, rellenarlo con guión bajo
- No esta habilitado el botón de grabación del microcontrolador en el MPLABX
 - No has conectado el Curiosity Nano, o el cable no tiene capacidad de transferencia de datos
 - Te has olvidado de escoger la herramienta (propiedades del proyecto: Tool)
- No se colorean las palabras en el código fuente.
 - Primero deben de grabar todos los archivos fuente presionando el botón de diskette.

Agenda:

- Estructura interna de un microcontrolador
- El módulo de oscilador
- Memoria de programa del microcontrolador PIC18F47Q10
- Memoria de datos del microcontrolador PIC18F47Q10
- Instrucciones básicas en XC8 PIC Assembler para el PIC18F47Q10
- ~~El contador de programa (PC) del CPU del microcontrolador PIC18F57Q43~~
- ~~Acceso a datos almacenados en la memoria de programa empleando el puntero del tabla (TBLPTR)~~
- ~~Acceso mediante punteros FSRx/INDFx a la memoria de datos~~
- ~~Interface a display de siete segmentos~~
- ~~Instrucciones de comparación numérica en XC8 PIC Assembler~~

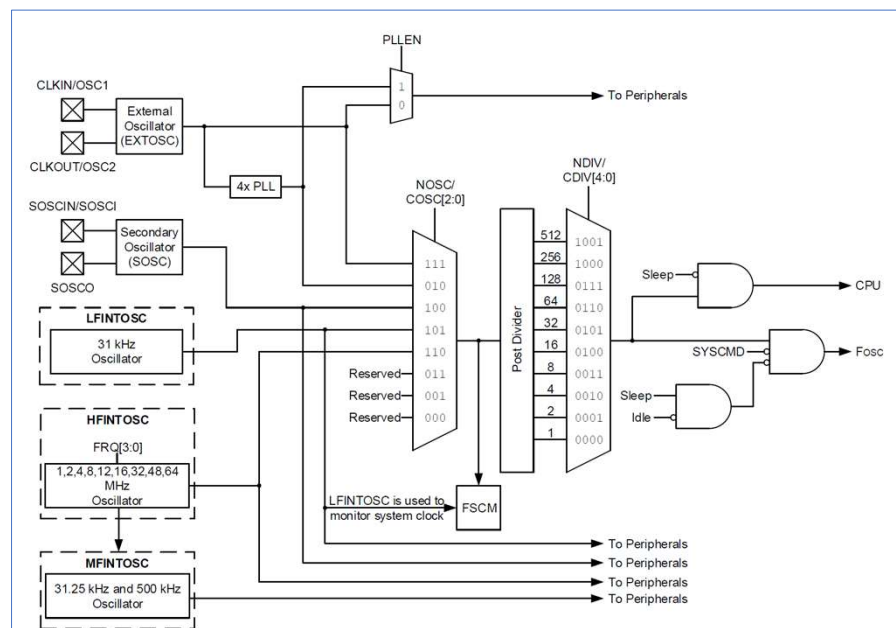
Asignación (10 mins)

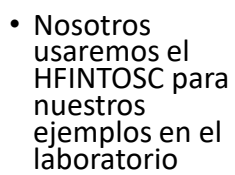
Hoja técnica del PIC18F47Q10:

- Revisar capítulo 5 – Módulo de oscilador
 - Diagrama de bloques figura 5-1
 - Apartado 5.5 Resumen de registros del módulo de oscilador
- Revisar capítulo 11 – Organización de las memorias
 - Memoria de programa 11.1
 - Memoria de datos 11.3
- Revisar capítulo 16 – Puertos de entrada/salida (E/S ó I/O)
 - Apartado 16.4 Resumen de registros de las E/S
- Revisar capítulo 38 – Resumen del set de instrucciones
 - Tabla 38-2
 - Apartado 38.1.1

El módulo de oscilador

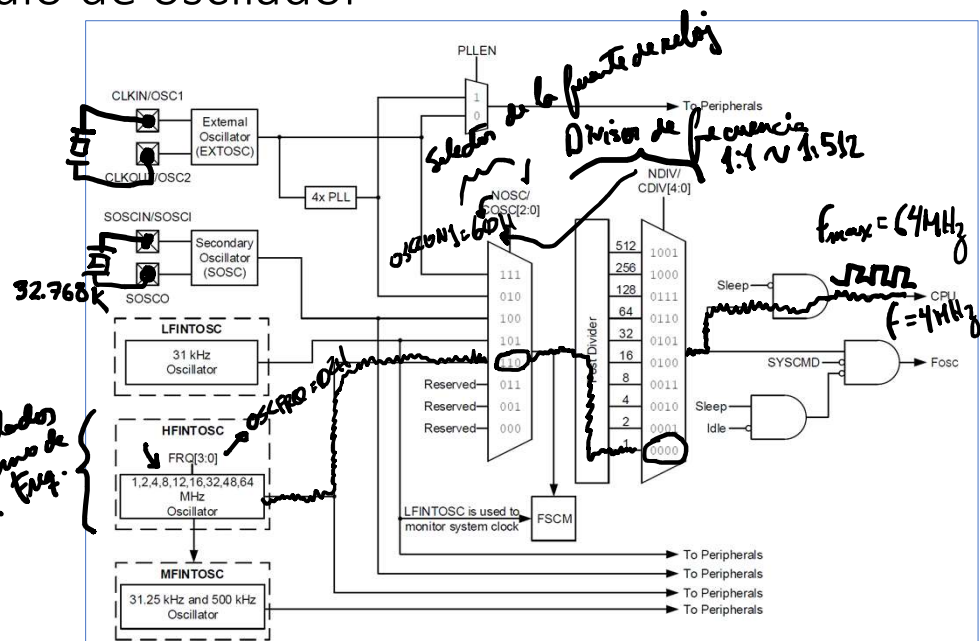
- Diagrama de bloques del módulo de oscilación del microcontrolador, obtenido de la hoja técnica.
- Aquí se configura la fuente de reloj que alimentará al CPU del microcontrolador y periféricos.
- Para aplicaciones muy críticas en lo que respecta a precisión de reloj, se recomienda usar cristal externo



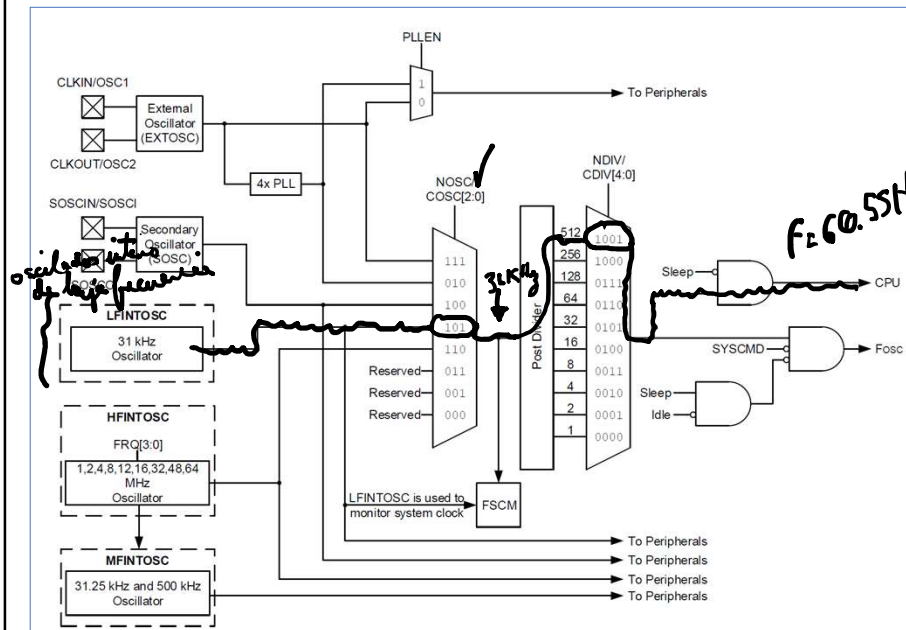


- La frecuencia a configurar en el HFINTOSC será de 4MHz
- MFINTOSC no se puede utilizar para alimentar al CPU

Osciladores
Internos de
Alta Freq.



Camilo: ¿Cuál es la mínima frecuencia posible?

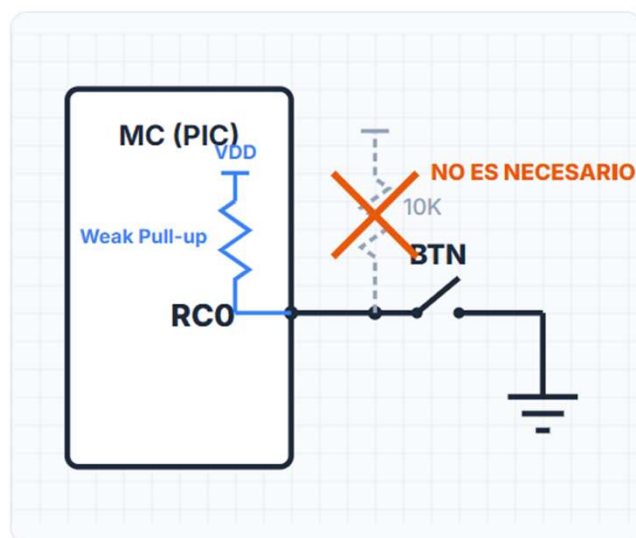


- Si la frecuencia que recibe el CPU es de 60.55Hz, cada instrucción simple en ensamblador se ejecutará en:

$$t_{ejec} = \left(\frac{F_{OSC}}{4} \right)^{-1} = 6.6 \text{ ms}$$

Recordando la configuración de los puertos de E/S:

- Definir las configuraciones al siguiente caso:



REGLA VISTA EN CLD (CIRCUITOS LÓGICOS DIGITALES)

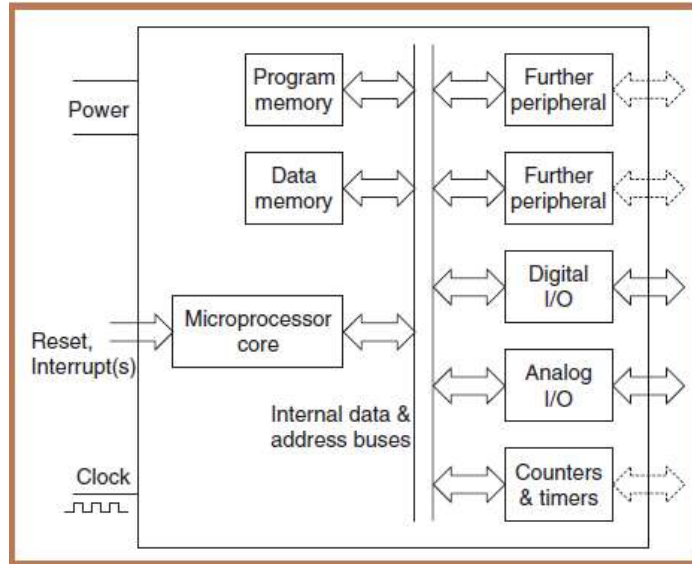
Las entradas no deben estar al aire.

Requisitos del Pin

- RCO es una entrada.
- RCO es digital.
- RCO habilita su weak pull-up.

```
XC8 PIC Assembler
bsf TRISC, 0, 1
bcf ANSEL, 0, 1
bsf WPUC, 0, 1
```

Estructura interna de un microcontrolador (introducción a la arquitectura de computadores)



- CPU: Encargado de ejecutar las instrucciones
- Soporte del CPU: módulo de reloj, dispositivos de seguridad, etc.
- **Memoria de programa: No volátil**, almacena programa y datos constantes
- **Memoria de datos: Volátil**, almacena datos temporales y contiene los registros SFR
- Entradas y salidas
- Periféricos

¿Cuál es la diferencia entre microprocesador y microcontrolador?

- **El microprocesador** es la unidad central de proceso (CPU) de un computador o un microcontrolador.
- **El microcontrolador** contiene todos los elementos para un funcionamiento autónomo, es decir, contiene un microprocesador, memorias y dispositivos de E/S.

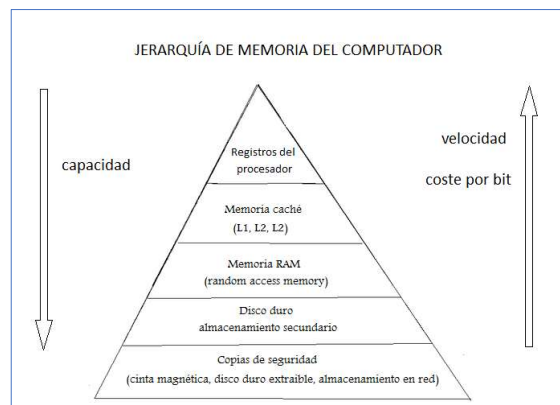
¿Y el Raspberry Pi qué cosa es, un microprocesador o un microcontrolador?



- No es un microcontrolador, tampoco es un microprocesador!
- Computador de Placa Reducida (SBC – Single Board Computer)

¿Por qué la memoria de datos siempre es del tipo volátil?

- La memoria de datos tiene que equiparar la velocidad de trabajo del CPU. La RAM es la que puede llegar a cumplir dicho requerimiento.



El Microcontrolador PIC18 de Microchip

Estructura de la memoria de **programa** del PIC18F47Q10:

- 128Kbyte de capacidad (000000H-01FFFFFFH)
- Data EEPROM (1Kbyte) se encuentra mapeado en 380000H
- Bits de configuración están mapeadas en 300000H – 300009H
- Rango de direcciones: 000000H-3FFFFFFH (22 bits – 4Mbyte de direcciones)
- Recordar que la dirección 000000H es el vector de RESET

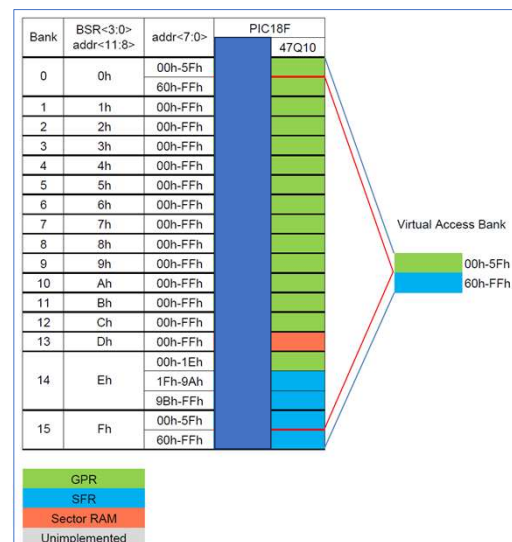
Address	Device
00 0000h to 00 3FFFh	PIC18F47Q10
00 4000h to 00 7FFFh	Program Flash Memory (64 KW) ⁽¹⁾
00 8000h to 00 FFFFh	
01 0000h to 01 FFFFh	
02 0000h to 1F FFFFh	
20 0000h to 20 003Fh	User ID (32 Words) ⁽²⁾
20 0040h to 2B FFFFh	Reserved
2C 0000h to 2C 00FFh	Device Information Area (DIA) ⁽³⁾
2C 0100h to 2F FFFFh	Reserved
30 0000h to 30 0009h	Configuration Bytes ⁽⁴⁾
30 000Ah to 37 FFFFh	Reserved
38 0000h to 38 03FFh	Data EEPROM (1024 Bytes)
38 0400h to 3B FFFFh	Reserved
3C 0000h to 3C 0009h	Device Configuration Information
3C 000Ah to 3F FFFFh	Reserved
3F 0000h to 3F 00FFh	Revision ID (1 Word) ^(5,6)
3F 0100h to 3F FFFFh	Device ID (1 Word) ^(5,6)

Notes: 1. Storage Area Flash is implemented as the last 128 Words of User Flash, if enabled.
 2. The addresses do not roll over. The region is read as '0'.
 3. Not code-protected.
 4. Hard-coded in silicon.
 5. This region cannot be written by the user and it is not affected by a Bulk Erase.

El Curiosity Nano PIC18F de Microchip

Estructura de la memoria de **datos** del PIC18F47Q10:

- Memoria del tipo volátil (se borra el contenido en un PoR – Power-on Reset)
- A diferencia del PIC18F57Q43, la memoria RAM (GPR) esta mapeada a partir del Bank 0 (000H) y los registros de funciones especiales (SFR) se encuentran entre Bank14 y Bank15 empezando desde la dirección E1FH
- Los SFR son los registros que permiten configurar algún recurso del microcontrolador (ej fuente de reloj, manipulación de las E/S)
- Tener en cuenta que la RAM de datos es de 3.6Kbyte
- El rango total de direcciones: 000H-FFFH (12 bits - 4096 direcciones)



Información del BSR

- Por defecto, al energizar o resetear el microcontrolador, el valor de BSR es 0000 (Bank0)

11.8.13 BSR

Name: BSR
Address: 0xFE0

bsr default value

Bank Select Register

The BSR indicates the data memory bank which is bits 11:8 of the GPR address.



Bits 3:0 – BSR[3:0]

Four Most Significant bits of the data memory address

Los registros SFR

- Registros de funciones especiales
- Sirven para configurar algún recurso del microcontrolador
- Los SFR se encuentran repartidos entre el Bank14 y Bank15
- No hay que olvidar de cambiar el banco para escribir ó leer algún registro SFR.

Bank	BSR<3:0> addr<11:8>	addr<7:0>	PIC18F
0	0h	00h-5Fh	47Q10
1	1h	60h-FFh	
2	2h	00h-FFh	
3	3h	00h-FFh	
4	4h	00h-FFh	
5	5h	00h-FFh	
6	6h	00h-FFh	
7	7h	00h-FFh	
8	8h	00h-FFh	
9	9h	00h-FFh	
10	Ah	00h-FFh	
11	Bh	00h-FFh	
12	Ch	00h-FFh	
13	Dh	00h-FFh	
14	Eh	00h-1Eh	
		1Fh-9Ah	
		9Bh-FFh	
15	Fh	00h-FFh	

Legend: GPR (Green), SFR (Blue), Sector RAM (Orange), Unimplemented (Grey)

Registros para configurar el módulo de oscilador

Bank 14
OSCCON1 → 60H
OSCFRQ → 02H
OSCCN → 40H

HFOEN = 1

NOSC = HFINTOSC

NDIV = 1:1

5.5 Register Summary - OSC

Address	Name	Bit Pos.	7	6	5	4	3	2	1	0
0x00	Reserved									
0x002	OSCCON1	7:0	0	1	0	0	0	0	0	0
0x003	OSCCON2	7:0	0	0	0	0	0	0	0	0
0x004	OSCCON3	7:0	0	0	0	0	0	0	0	0
0x005	OSCCON4	7:0	0	0	0	0	0	0	0	0
0x006	OSCCON5	7:0	0	0	0	0	0	0	0	0
0x007	OSCCON6	7:0	0	0	0	0	0	0	0	0
0x008	OSCCON7	7:0	0	0	0	0	0	0	0	0
0x009	OSCCON8	7:0	0	0	0	0	0	0	0	0

Handwritten notes: HFOEN = 1, NOSC = HFINTOSC, NDIV = 1:1, FREQ = 4MHz

Los registros SFR

NOTA: Los puertos por defecto en un reset o energizado están como entradas analógicas

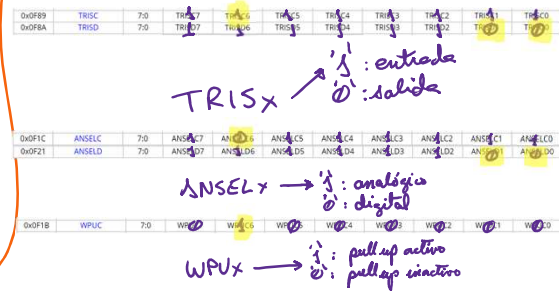
- Los registros de manipulación de los puertos de E/S se encuentra en el Bank15
- Son varios registros para manipular los puertos, recordando que tenemos desde la A hasta la E

Bank	BSR<3:0> addr<11:8>	addr<7:0>	PIC18F
0	0h	00h-5Fh	47Q10
1	1h	60h-FFh	
2	2h	00h-FFh	
3	3h	00h-FFh	
4	4h	00h-FFh	
5	5h	00h-FFh	
6	6h	00h-FFh	
7	7h	00h-FFh	
8	8h	00h-FFh	
9	9h	00h-FFh	
10	Ah	00h-FFh	
11	Bh	00h-FFh	
12	Ch	00h-FFh	
13	Dh	00h-FFh	
14	Eh	00h-1Eh 1Fh-9Ah 9Bh-FFh	
15	Fh	00h-5Fh 60h-FFh	

- PORTx registers (reads the levels on the pins of the device)
- LATx registers (output latch)
- TRISx registers (data direction)
- ANSELx registers (analog select)
- WPUx registers (weak pull-up)

GPR
SFR
Sector RAM
implemented

Necesitamos RDD y RDI que sean salida digital y RCG que sea entrada digital con pull-up



La importancia de escoger el banco correcto en la memoria de datos

- Los registros ó direcciones en la memoria de datos se encuentran clasificados en bancos (Bank) que son los bits mas significativos (bit 8 al bit 13) del valor de dirección.
- Es importante seleccionar el banco correcto antes de manipular un registro en la memoria de datos, revisar la hoja técnica para ver en qué banco se encuentra el registro a manipular.
- Por ejemplo: Deseo manipular el puerto RE0 como salida digital

Ejemplo para PIC18F47Q10:

```
movlb 0FH      ;me voy al Bank15
bcf TRISE, 0, 1 ;RE0 sea salida
bcf ANSELE, 1, 1 ;RE0 sea digital
```

Recordando: Estructuras anidadas

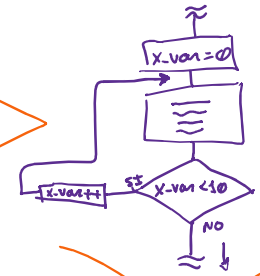
En el lenguaje C:

```

      8 bits
unsigned char x_var = 0;
for (x_var = 0; x_var < 10; x_var++) {
    //
}
    
```

límite → 255

se va a repetir 10 veces



Cómo hago para hacer mas repeticiones teniendo como limitante el tipo de variable?

```

unsigned char x_var = 0, y_var = 0;
for (x_var = 0; x_var < 200; x_var++) {
    for (y_var = 0; y_var < 200; y_var++) {
        //
    }
}
    
```

¿cuántas veces se repite? → 40000

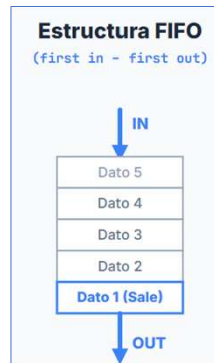
XC8 PIC Assembler

Recordando saltos a sub-rutinas

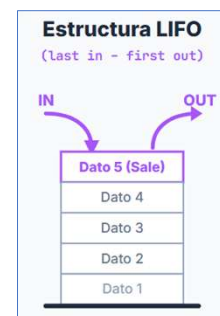
- ¿Cómo sabe el CPU que debe de retornar de donde saltó?
 - Existe una memoria el cual almacena la dirección de retorno y que le va a permitir al CPU regresar una vez atendido la eventualidad.
 - Dicha memoria se le conoce como "stack" o de pila (acción de apilar).
- En el XC8 PIC Assembler se tiene CALL-RETURN

Memorias “Stack” o de pila (apilamiento ó acción de apilar)

- Estructura FIFO
(first in – first out)



- Estructura LIFO
(last in – first out)



Aspectos relacionados con el obsoleto MPASM y el actual XC8 PIC Assembler

- El año 2014 Microchip dejó de lado el MPASM para dar lugar al XC8 PIC Assembler (PIC-AS)
- Nuevos diseños deberán emplean XC8 PIC Assembler en lugar de MPASM.
- XC8 PIC Assembler es un lenguaje de bajo nivel (orientado a la máquina), **nosotros debemos de conocer primero cómo funciona la máquina** para luego hacer que funcione mediante la codificación de un programa en Assembler y dar solución al problema planteado.

MPASM vs XC8 PIC Assembler: Partes de un programa

MPASM:

```
list p=18f4550
#include<pl18f4550.inc>

;aquí declaramos los bits
CONFIG FOSC = XT_XT
CONFIG FWRT = ON
CONFIG BOR = OFF
CONFIG WDT = OFF
CONFIG PBADEN = OFF
CONFIG LVP = OFF

org 0x0000
goto configur0

org 0x0020
configur0:
    bsf TRISB, 0
    bcf TRISD, 0

principal:
    btfss PORTB, 0
    goto principal
    btg LATD, 0

otro:
    btfsc PORTB, 0
    goto otro
    goto principal

end
```

XC8 PIC ASM:

```
PROCESSOR 18F4550
#include "cabecera.inc"

PSECT rstVect,class=CODE, reloc=2, abs
ORG 0000H

rstVect:
    goto configur0
    ORG 0020H

configur0:
    bsf TRISB, 0, 0
    bcf TRISD, 0, 0

principal:
    btfss PORTB, 0
    goto principal
    btg LATD, 0, 0

otro:
    btfsc PORTB, 0
    goto otro
    goto principal

end rstVect
```

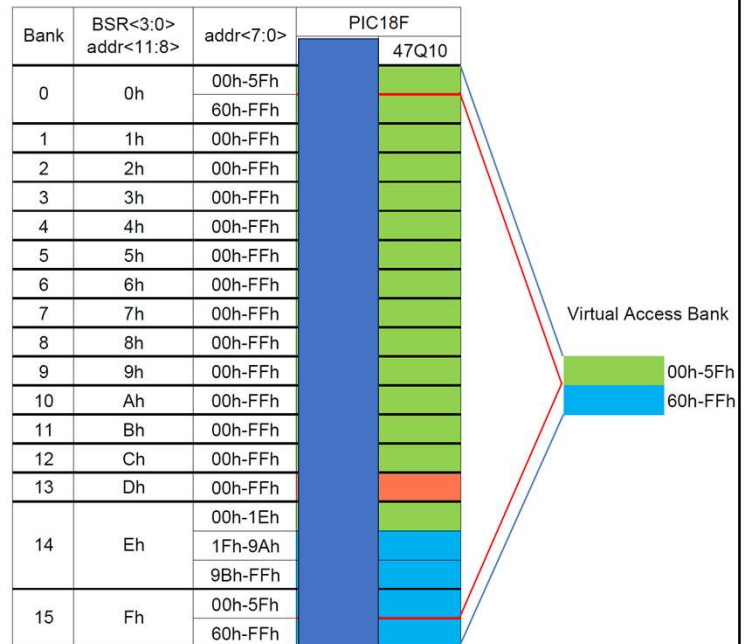
Nota: Los bits de configuración se alojaron en un archivo header llamado "cabecera.inc"

¿Por qué aprender Assembler (más difícil) si en el C también se puede hacer?

- En el assembler se prioriza en la eficiencia (espacio empleado en la memoria de programa y de datos, control detallado del consumo energético y del funcionamiento en general)
- En el C se prioriza en menor tiempo de Desarrollo
- Hay que recordar que el uso de lenguaje de alto nivel siempre será menos eficiente frente al assembler (va a ocupar mas espacio, lo vas a ver como caja negra al microcontrolador)

Access-Bank vs BSR

- Son las formas para acceder a la memoria de datos
- Access-Bank (param a=0)** es una forma directa (pero limitada) de acceder a la memoria de datos, se usa para tener acceso a una porción del Bank15 (60H-FFH) y una porción de memoria RAM en Bank0 (00H-5FH) y evitar estar cambiando de bancos de manera frecuente
- BSR (Bank Select Register, param a=1)** es la forma estándar para acceder a la memoria de datos, previamente se tiene que seleccionar el banco de entre 0 y 15 usando el registro selector de bancos BSR ó usando directamente la instrucción "movlb"



Repertorio de instrucciones en Assembler del PIC18

- Revisar capítulo 38 de la hoja técnica del PIC18F47Q10

Table 44-2. Standard Instruction Set

Mnemonic, Operands	Description	Cycles	16-Bit Instruction Word				Status Affected	Notes
			MSb			LSb		
BYTE-ORIENTED FILE REGISTER INSTRUCTIONS								
ADDWF f, d, a	Add WREG and f	1	0010	01da	ffff	ffff	C, DC, Z, OV, N	1
ADDFWC f, d, a	Add WREG and Carry bit to f	1	0010	00da	ffff	ffff	C, DC, Z, OV, N	1
ANDWF f, d, a	AND WREG with f	1	0001	01da	ffff	ffff	Z, N	1
CLRF f, a	Clear f	1	0110	101a	ffff	ffff	Z	
COMF f, d, a	Complement f	1	0001	11da	ffff	ffff	Z, N	1
DECf f, d, a	Decrement f	1	0000	01da	ffff	ffff	C, DC, Z, OV, N	1
INCF f, d, a	Increment f	1	0010	10da	ffff	ffff	C, DC, Z, OV, N	1
IORWF f, d, a	Inclusive OR WREG with f	1	0001	00da	ffff	ffff	Z, N	1
MOVF f, d, a	Move f to WREG or f	1	0101	00da	ffff	ffff	Z, N	1
MOVWF f _s , f _d	Move f _s (12-bit source) to f _d (12-bit destination)	2	1100	f _s f _s f _s f _s f _d f _d f _d f _d	f _s f _s f _s f _s f _d f _d f _d f _d	f _s f _s f _s f _s f _d f _d f _d f _d	None	1, 3, 4
MOVFL f _s , f _d	Move f _s (14-bit source) to f _d (14-bit destination)	3	1111	f _s f _s f _s f _s f _s f _s f _s f _s	f _s f _s f _s f _s f _s f _s f _s f _s	f _s f _s f _s f _s f _s f _s f _s f _s	None	1, 3
MOVWF f, a	Move WREG to f	1	0110	111a	ffff	ffff	None	
MULWF f, a	Multiply WREG with f	1	0000	001a	ffff	ffff	None	1
NEGf f, a	Negate f	1	0110	110a	ffff	ffff	C, DC, Z, OV, N	1
RLCF f, d, a	Rotate Left f through Carry	1	0011	01da	ffff	ffff	C, Z, N	1
RLNCF f, d, a	Rotate Left f (No Carry)	1	0100	01da	ffff	ffff	Z, N	1
RRCF f, d, a	Rotate Right f through Carry	1	0011	00da	ffff	ffff	C, Z, N	1
RRNCF f, d, a	Rotate Right f (No Carry)	1	0100	00da	ffff	ffff	Z, N	1
SETF f, a	Set f	1	0110	100a	ffff	ffff	None	
SUBRWB f, d, a	Subtract f from WREG with Borrow	1	0101	01da	ffff	ffff	C, DC, Z, OV, N	1
SUBWF f, d, a	Subtract WREG from f	1	0101	11da	ffff	ffff	C, DC, Z, OV, N	1
SUBWFB f, d, a	Subtract WREG from f with Borrow	1	0101	10da	ffff	ffff	C, DC, Z, OV, N	1
SWAPF f, d, a	Swap nibbles in f	1	0011	10da	ffff	ffff	None	1
XORWF f, d, a	Exclusive OR WREG with f	1	0001	10da	ffff	ffff	Z, N	1

Repertorio de instrucciones en Assembler del PIC18

- Revisen las instrucciones que se han empleado en el ejemplo de la semana pasada con esta tabla de instrucciones:
 - movlb -> seleccionar el banco de trabajo, ej movlb 0H ;vas al Bank0
 - movlw -> mover un literal hacia el registro W, ej movlw 60H
 - movwf -> mover el contenido de W hacia un registro (memoria de datos)
 - bsf -> setear un bit de un registro (poner a uno lógico)
 - bcf -> resetear un bit de un registro (poner a cero lógico)
 - btfss -> preguntar si un bit de un registro es igual a uno
 - bra -> salto incondicional, ej bra inicio

Detalle de una instrucción con opción {a}

• Ejemplo:

Trabajando con Access bank:
movwf TRISB, 0

Trabajando con BSR:

(previamente especificar en el BSR cuál es el banco de acceso)

movwf TRISB, 1

movwf TRISB, a

movwf TRISB, b

← access bank

← BSR

- No todas las instrucciones tienen el parámetro {a}, revisar capítulo 44 del datasheet

MOVWF	Move W to f			
Syntax	MOVWF f [,a]			
Operands	0 ≤ f ≤ 255 a ∈ [0, 1]			
Operation	(W) → f			
Status Affected	None			
Encoding	0110	111a	ffff	ffff
Description	Move data from W to register 'f'. Location 'f' can be anywhere in the 256-byte bank. If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank. If 'a' is '0' and the extended instruction set is enabled, this instruction operates in Indexed Literal Offset Addressing mode whenever f ≤ 95 (5Fh). See Byte-Oriented and Bit-Oriented Instructions in Indexed Literal Offset Mode for details.			
Words	1			
Cycles	1			

Q Cycle Activity:

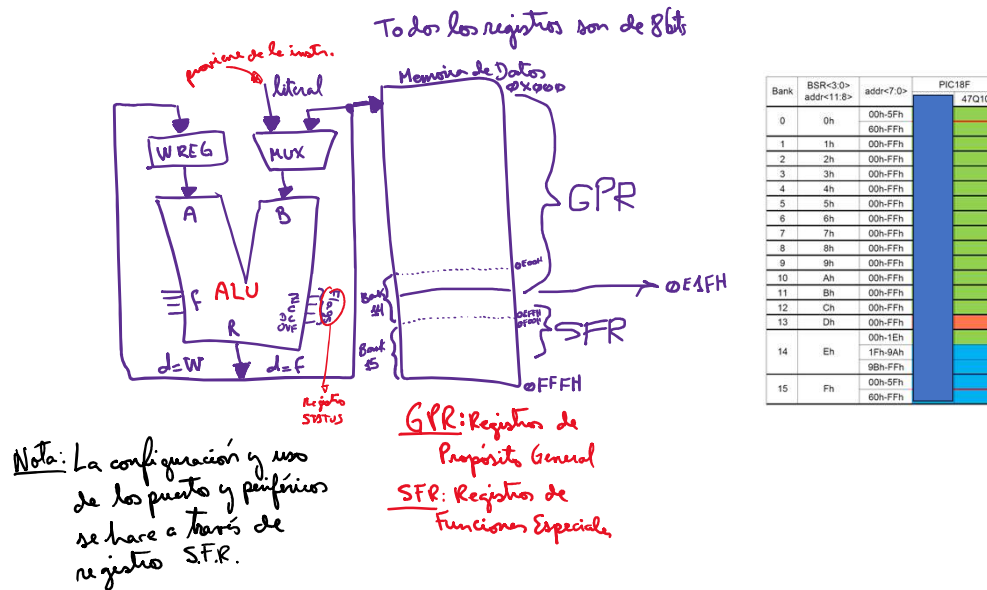
Q1	Q2	Q3	Q4
Decode	Read W	Process Data	Write register 'f'

Example: MOVWF REG, 0

Before Instruction
W = 4Fh
REG = FFh

After Instruction
W = 4Fh
REG = 4Fh

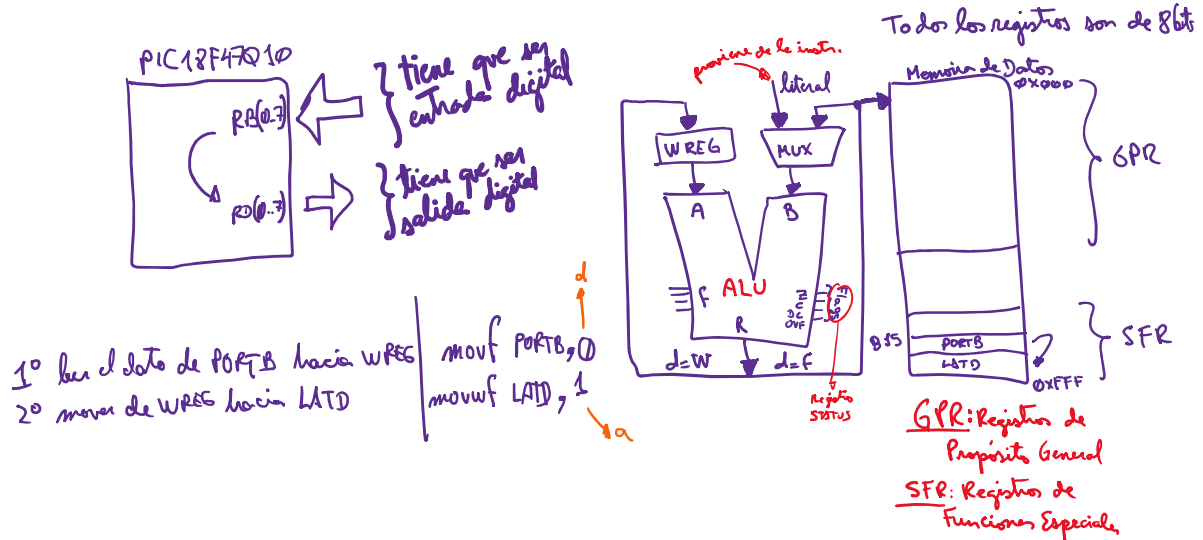
El flujo de datos en el microcontrolador PIC18F47Q10



Comentarios y Notas:

- En los programas desarrollados en XC8 PIC Assembler, la mayor cantidad de instrucciones serán las de movimiento de datos.
- Es preferible trabajar con BSR (a=1), debido a que en el modo Access Bank solo se tiene una parte de los registros del Bank0 y una parte de GPR del Bank15, si tu quieres emplear algún otro registro fuera de ese grupo, igual tendrás que el BSR.

Ejemplo: Pasar un dato del puerto B al puerto D



Registro STATUS

- Encuentras las banderas de la ALU, se actualizan cuando ocurre alguna operación aritmética ó lógica en este dispositivo.

7.7.7 STATUS

Name: STATUS
Address: 0x4D8

STATUS Register

Bit	7	6	5	4	3	2	1	0
Access		R	R	R/W	R/W	R/W	R/W	R/W
Reset		1	1	0	0	0	0	0

Handwritten note: "de la ALU" (from the ALU) with a bracket under bits 4, 3, 2, 1, and 0.

Bandera "N": Cuando en una operación que realiza la ALU el resultado fué un número negativo

Bandera "OV": Cuando ocurre un desbordamiento del dato que se ha incrementado

Bandera "Z": Cuando en una operación que realiza la ALU el resultado salió cero (00000000B ó 00H ó 0D)

Bandera "DC": El digit carry

Bandera "C/~B": Bit carry/~borrow, empleado en las operaciones aritméticas de suma y resta

Instrucciones básicas en XC8 PIC Assembler

- movlb -> para establecer el banco donde se va a trabajar
- movlw -> para mover un literal hacia el registro Wreg
- movwf -> para mover contenido del Wreg hacia un registro
- movff -> para mover el contenido de un registro a otro registro
- bsf, bcf -> para colocar 1 ó 0 a un bit (7-0) de un registro
- btfss, btfsc -> para probar si un bit es uno ó cero
- nop -> no operación (pierde el tiempo según t_{ejec})
- decf, incf -> para decrementar/incrementar el valor de un registro
- incfsz, decfsz -> incremento/decremento con pregunta si llegó a cero
- setf, clrf -> coloca todos a uno/cero los bits de un registro
- comf -> complementa todos los bits de un registro
- bra, goto -> saltos, bra = cortos, goto = largos
- call, return -> saltos a subrutina (con opción a retornar)

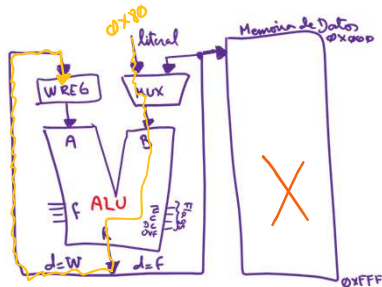
Instrucciones básicas en XC8 PIC Assembler

- Instrucciones de movimiento de datos

movlw [literal]

-mover un literal hacia WREG

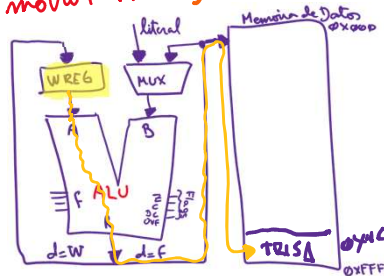
Ej: *movlw 80H*



movwf [registro], a

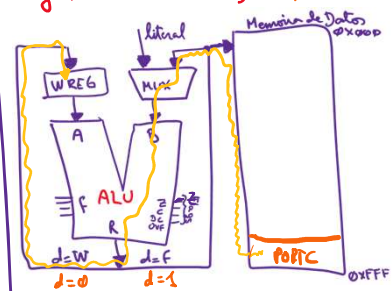
-mover el contenido de WREG hacia [registro]

Ej: *movwf TRISA, 1*



movf [registro], d, a
-mover el contenido de [registro] y lo mueve según "d".

Ej: *movf PORTC, 0, 1*



Nota: *movf [registro], f* sirve para act. flag

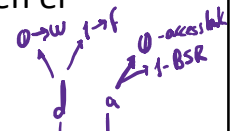
¿Instrucción movfw?

- ¡Instrucción no documentada en la hoja técnica!
- Instrucción “legacy”, instrucción de compatibilidad
- Esta no es una instrucción en sí, sino una “pseudo-instrucción” del lenguaje assembler.
- Lo que hace es mover el contenido de un registro y lo coloca en el Wreg.

Ej: `movfw TRISA`

simila

`movf TRISA, 0, 1`



Instrucciones básicas en XC8 PIC Assembler

- Instrucciones de manipulación de un bit determinado, en un registro

`bsf [registro], #bit, posición`
coloca a "1" el #bit de [registro]

Ej:

`bsf TRISB, 3, 1`
quiero ponerlo a "1"

TRISB
1 0 1 1 1 0 0 1 1

`bcf [registro], #bit, posición`
coloca a "0" el #bit de [registro]

Ej:

`bcf TRISB, 7, 1`

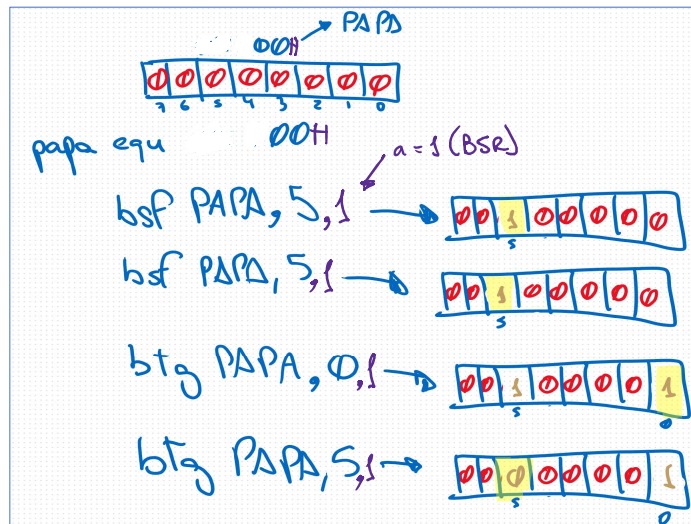
TRISB
0 0 1 1 1 0 0 1 1

`btg [registro], #bit, posición`
aplica complemento al #bit de [registro]

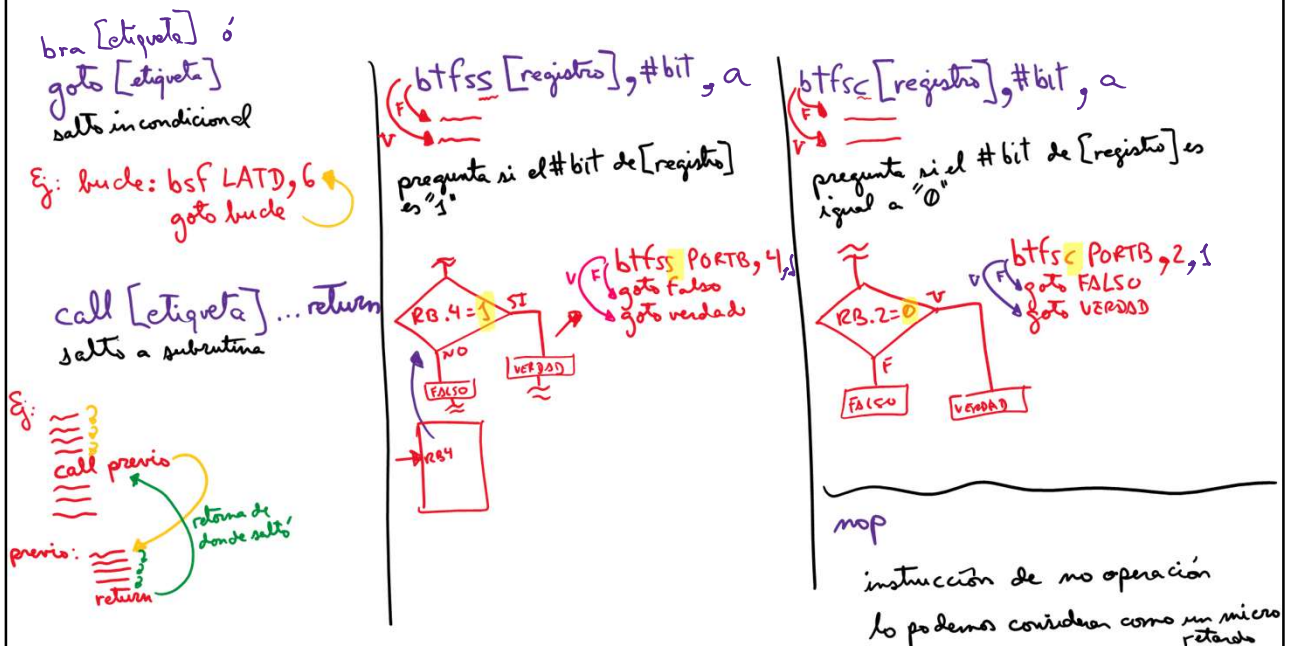
Ej: `btg TRISB, 6, 1`

TRISB
1 1 1 1 1 0 0 1 1
↓
0

Ejemplo de uso de instrucciones de manipulación de bits en un registro



Instrucciones básicas en XC8 PIC Assembler



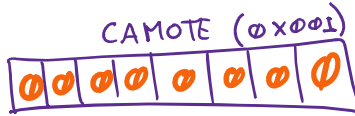
Instrucciones básicas en XC8 PIC Assembler

- Instrucción decf / incf
 - Decremento (decf) o incremento (incf) de registro, ambos de **uno en uno**

Ej: `decf [registro], d, a` `incf [registro], d, a`
"d" puede ser: f ó w

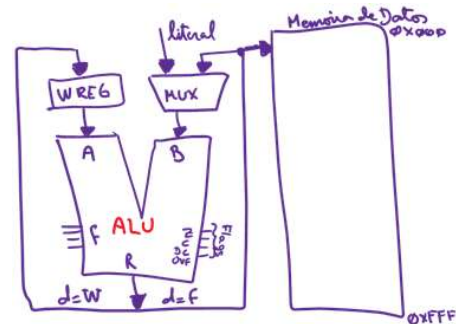


`decf CAMOTE, f`



`decf CAMOTE, f`

→ Registro STATUS: Z = 1



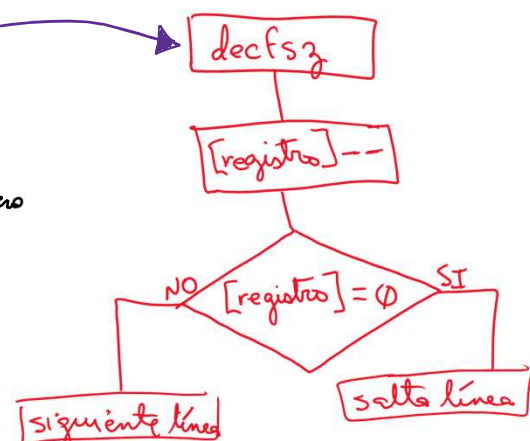
Instrucciones básicas en XC8 PIC Assembler

`decfsz [registro], d`

decrementa y pregunta si [registro] llegó a cero

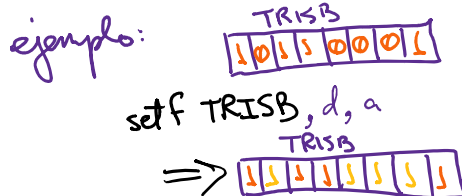
`incfsz [registro], d`

incrementa y pregunta si [registro] llegó a cero

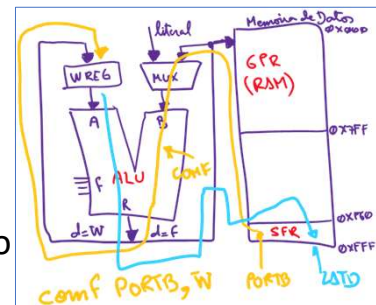
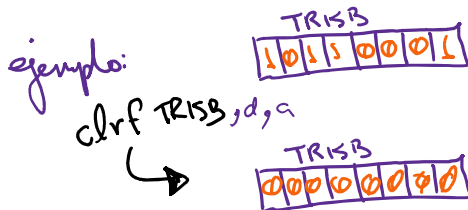


Instrucciones setf [registro], clrf [registro], comf [registro]

- **setf [registro]** : Coloca todos los bits del registro indicado a uno lógico



- **clrf [registro]** : Coloca todos los bits del registro indicado a cero lógico



- **comf [registro]** : Complementa todos los bits del registro

Ejemplo: Puerto D todos sus bits como salida digital:

Opción 1: usando bcf

```
bcf TRISD, 0, 1
bcf TRISD, 1, 1
⋮
bcf TRISD, 7, 1
```

↑
ocho instrucciones

```
bcf ANSELD, 0, 1
⋮
```

Opción 2: usando valor numérico

```
movlw 00H
movwf TRISD, 1
```

↑
una sola instrucción

```
movwf ANSELD, 1
```

Opción 3: usando clrf

```
clrf TRISD, 1
```

↑
una sola instrucción

```
clrf ANSELD, 1
```

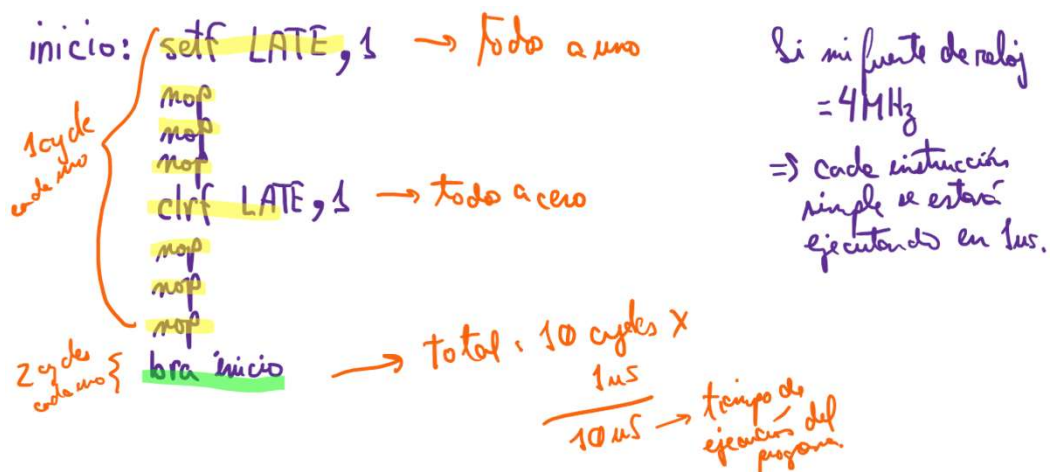
Tiempo de ejecución de las instrucciones en XC8 PIC Assembler

- Se utiliza la siguiente fórmula:

$$t_{opc} = \left(\frac{f_{osc}}{4} \right)^{-1} s$$

- Hay instrucciones simples, dobles y especiales (revisar 44.0 de la datasheet)
- Recordando la relación periodo vs frecuencia: $f = \frac{1}{T}$

Ejemplo de tiempo de ejecución



Ejemplo: Señal de cruce de tren

- El circuito tiene dos LEDs y deben de parpadear de forma alternada con un periodo de 500ms

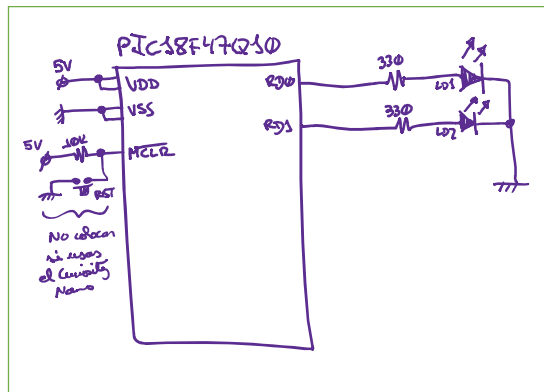


Diagrama esquemático

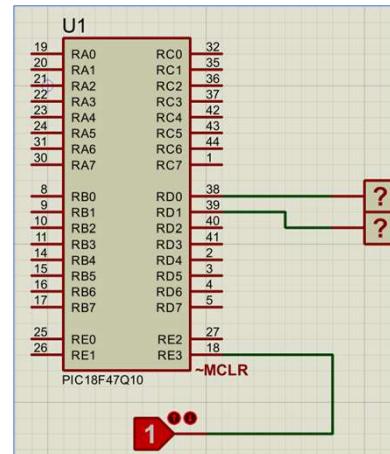
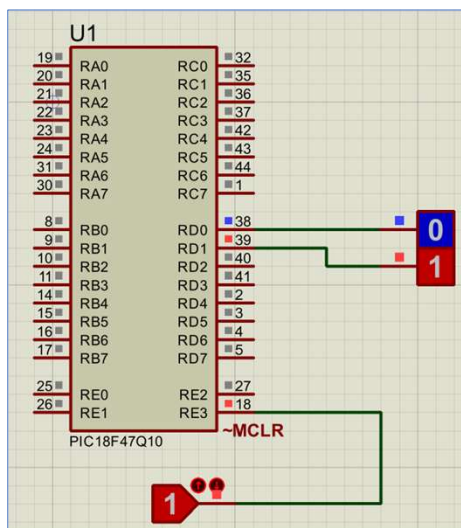


Diagrama en Proteus

Ejemplo: Señal de cruce de tren

- Código y ejecución de la simulación en Proteus

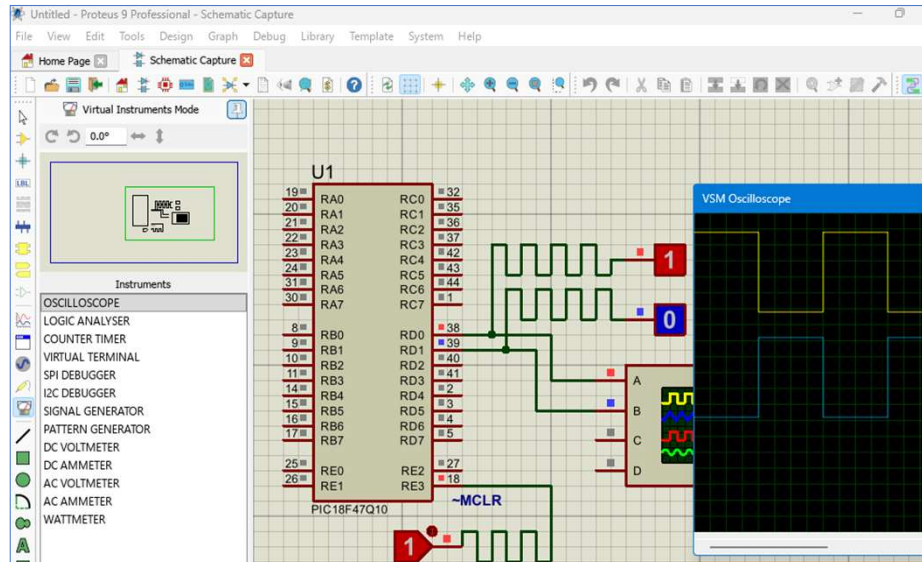


```

1  PROCESSOR 18F47Q10
2  #include "cabecera.inc"
3
4  PSECT upcinos, class=CODE, reloc=2, abs
5  upcinos:
6  ORG 000000H
7  bra configuro
8
9  ORG 000070H
10 configuro:
11 movlb 0EH
12 movlw 60H
13 movwf OSCCON1, 1
14 movlw 02H
15 movwf OSCFRQ, 1
16 movlw 40H
17 movwf OSCEN, 1
18 movlb 0FH
19 movlw 0FCH
20 movwf TRISD, 1
21 movwf ANSEL, 1
22
23 inicio:
24 movlw 01H ;00000001
25 movwf LATD, 1
26 bucle:
27 comf LATD, 1, 1 ;aplica el complemento a LATD
28 bra bucle
29
30 end upcinos
  
```

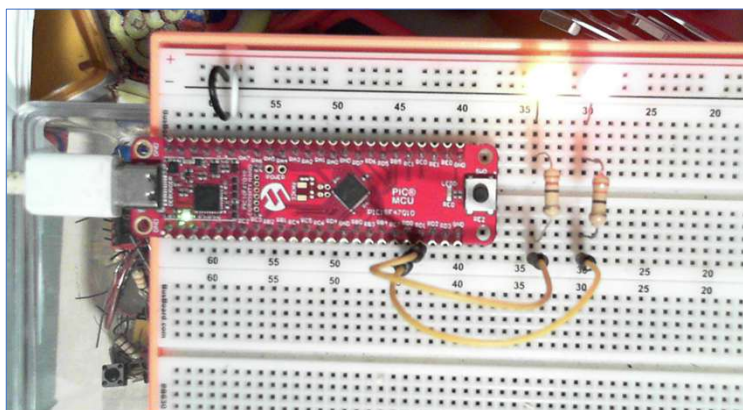
Ejemplo: Señal de cruce de tren

- Simulación en Proteus: visualización de señales en osciloscopio



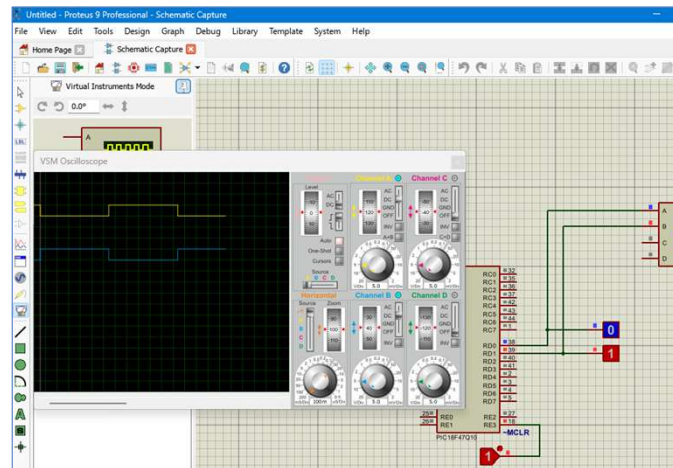
Ejemplo: Señal de cruce de tren

- Implementación física empleando en módulo de desarrollo Curiosity Nano PIC18F47Q10. Nótese que no se puede apreciar la alternancia debido a que el periodo de retardo es un corto al usar un solo nop.
- ¿Qué solución podemos darle para que se pueda apreciar la alternancia?
 - Aminorando la frecuencia de trabajo del CPU
 - Aplicando una cantidad considerable de instrucciones nop en cada alternancia



Minilab 2 (Semana 3) Turno Jueves 07:00

- Resultado de la simulación aplicando la primera estrategia de aminorar la frecuencia del CPU a 60Hz usando el LFINTOSC (31KHz) y el NDIV a 1:512. También adicionando algunos nops en el código.

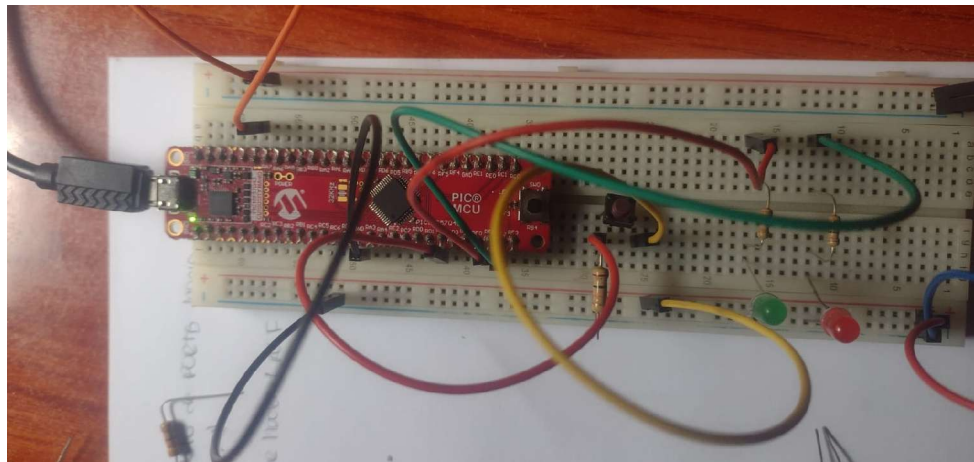


Ejercicios pasados

- Para que valides fuera de horario de clases:

Error en la implementación

- “Profesor compila pero no funciona”
- Ubica el error:

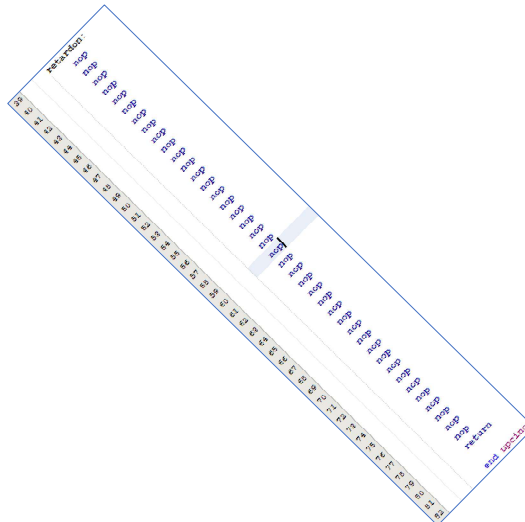


Problemas encontrados

- No funciona bien, parece que hay pulsaciones falsas en el botón
- Los botones tienen problema de rebote, por lo tanto se tienen que implementar una estrategia “anti-rebote”

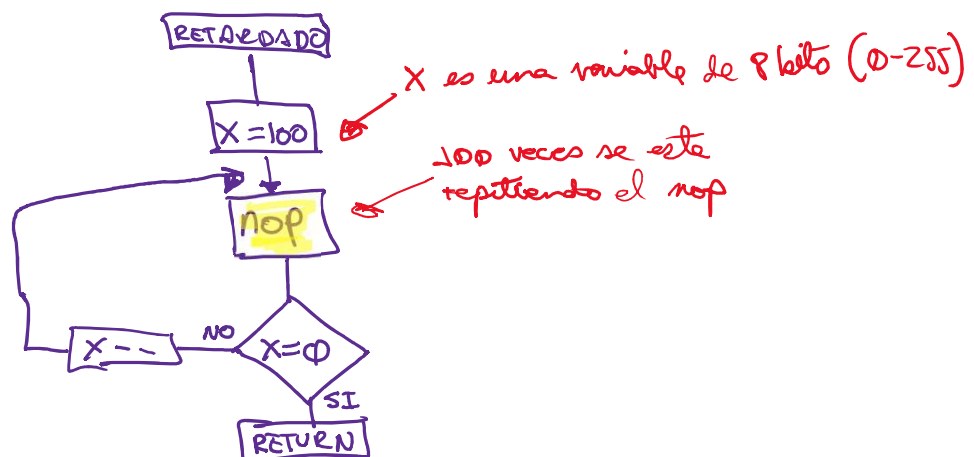
¿Cómo hago el antirrebote del botón sin tener que escribir tanto “nops”?

- Escribir 40000 nops no es práctico ni eficiente!



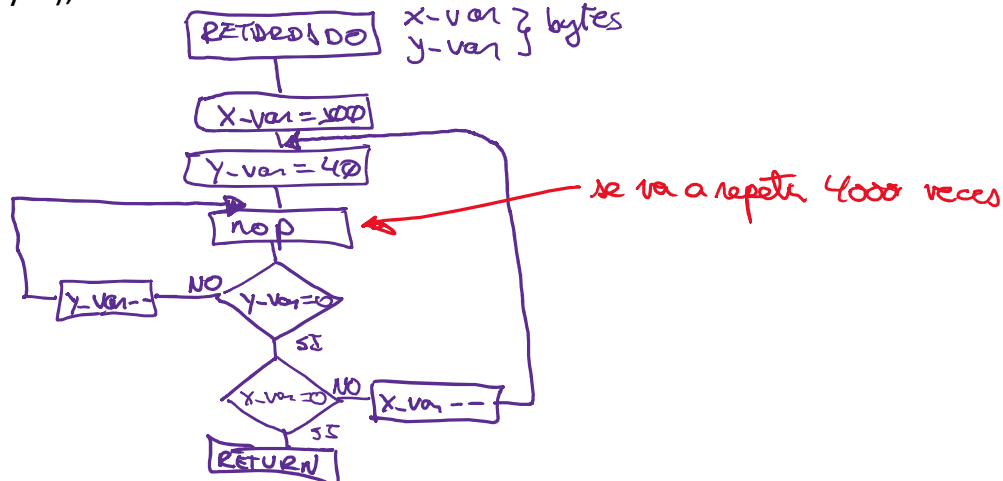
¿Cómo generamos el retardo de 40ms para el antirrebote del botón?

- Usar muchos nops ocupa demasiado espacio
- Se puede armar una rutina de bucle de repetición (for)



¿Cómo generamos el retardo de 40ms para el antirrebote del botón?

- Con un solo bucle de repetición solo llegamos a 255 (valor máximo en un byte), debemos de hacer "for" anidado



¿Cómo generamos el retardo de 40ms para el antirrebote del botón?

- x_var** y **y_var** son etiquetas asignadas a posiciones GPR de la memoria de datos, recordando que para el PIC18F47Q10 las GPR empiezan desde la dirección 000H.
- El código fue obtenido del diagrama de flujo anterior, tener en cuenta que al usar BSR se tiene que estar cambiando de bancos ya que el registro STATUS se encuentra en el BANK4 (4D8H) y los GPRs **x_var** y **y_var** están ubicados en BANK5 (500H y 501H respectivamente). Deberá de modificarse dichos valores de banco de emplearse el PIC18F47Q10.
- Reemplazarlo en los códigos anteriores para obtener un mejor filtro antirrebote

```

40  retardado:
41      movlb 5H
42      movlw 100
43      movwf x_var, 1
44  otro3:
45      movlw 40
46      movwf y_var, 1
47  otro2:
48      nop                                ;se va a ejecutar 40000 veces
49      movf y_var, 1, 1
50      movlb 4H
51      btfss STATUS, 2, 1
52      bra y_var_noeszero
53      movlb 5H
54      movf x_var, 1, 1
55      movlb 4H
56      btfss STATUS, 2, 1
57      bra x_var_noeszero
58      return
59  y_var_noeszero:
60      movlb 5H
61      decf y_var, 1, 1
62      bra otro2
63  x_var_noeszero:
64      movlb 5H
65      decf x_var, 1, 1
66      bra otro3
  
```

Ejercicios complementarios de manipulación de puertos con el PIC18F47Q10:

Tener en consideración el procedimiento para el desarrollo de los ejercicios (análisis del problema y requerimientos, diseño de hardware, diagrama de flujo, código en XC8 PIC Assembler y pruebas)

1. Colocar LEDs en todos los pines de RD y de RB (con su respectiva resistencia de 330Ω). Escribir 5AH en RD y 0A5H en RB.
2. Implementar una XOR de un bit empleando RD0 y RD4 como entradas y RB2 como la salida.
3. Recibir un dato de 8 bits en RD y replicarlo en complemento por RB
4. Con el algoritmo antirrebote basado en la generación de retardo de 40ms con bucles anidados visto en el último ejemplo 04. Ampliar dicho retardo hasta 500ms aproximadamente y realizar una aplicación de parpadeo de un LED conectado en el RA0.

Recomendaciones al momento de implementar el circuito en físico con el Curiosity Nano PIC18F47Q10:

- Verificar continuidad en los cables jumper antes de ser utilizados en el circuito.
- Verificar que el cable USB-microUSB tenga capacidad de transferencia de datos.
- Verificar que la PC haya detectado correctamente el Curiosity Nano PIC18F47Q10.
- Verificar que el proyecto creado en el MPLABX se haya seleccionado el Curiosity Nano PIC18F47Q10 (ventana de propiedades del proyecto y "connected hardware tool")
- No olvidar que el header file tiene extensión *.inc y el source file tiene extensión *.s
- Revisar que se haya configurado el Curiosity Nano PIC18F47Q10 para que entregue voltaje de 5V a través del pin VTG (ventana de propiedades del PKOB nano dentro de propiedades del proyecto y opción Power)
- Revisar siempre los mensajes en la ventana de "output" por posibles fallos en el evento de compilación y/o evento de programación.
- Tener a la mano un multímetro para verificar voltajes y continuidad de conexiones en el prototipo implementado

Ejercicio 2026-0:

- Al ejercicio del auto fantástico agregarle tres efectos luminosos adicionales (total cuatro), el cambio se hará mediante un solo pulsador ubicado en RB0.

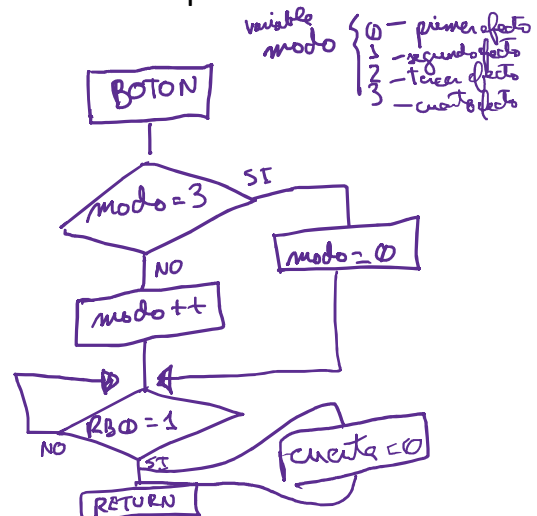
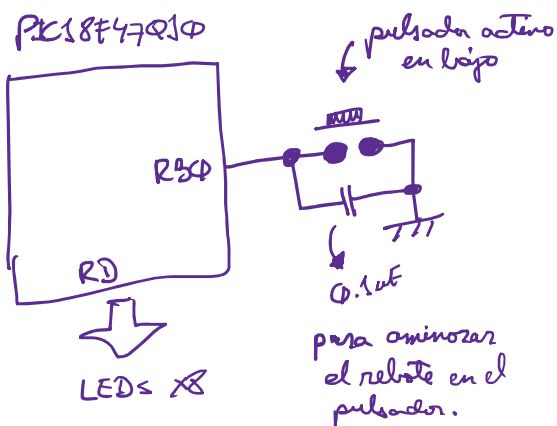
	L0	L1	L2	L3	L4	L5	L6	L7	
T0									03H
T1									06H
T2									0CH
T3									18H
T4									30H
T5									60H
T6									0C0H
T7									60H
T8									30H
T9									18H
T10									0CH
T11									06H

	L0	L1	L2	L3	L4	L5	L6	L7	
T0									07H
T1									0EH
T2									1CH
T3									38H
T4									70H
T5									0E0H
T6									70H
T7									38H
T8									1CH
T9									0EH

	L0	L1	L2	L3	L4	L5	L6	L7	
T0									81H
T1									42H
T2									24H
T3									18H
T4									24H
T5									42H

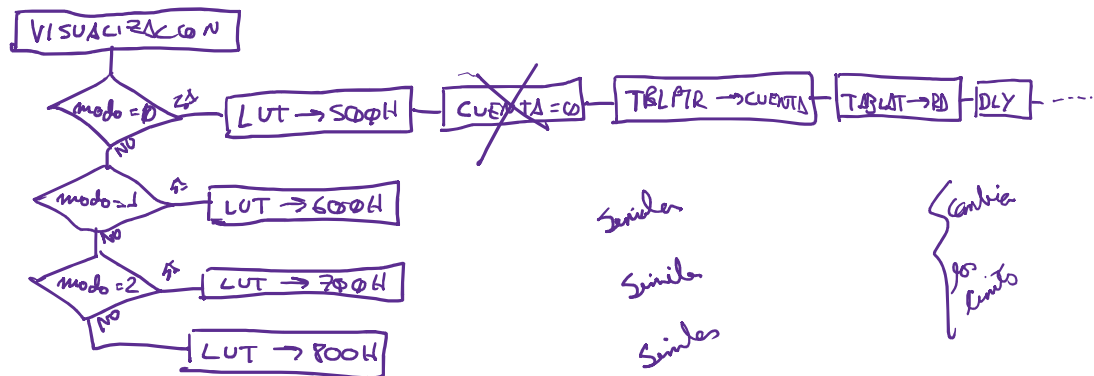
Ejercicio 2026-0:

- Al ejercicio del auto fantástico agregarle tres efectos luminosos adicionales (total cuatro), el cambio se hará mediante un solo pulsador ubicado en RB0.
- Funcionamiento del pulsador en RB0:



Ejercicio 2026-0:

- Al ejercicio del auto fantástico agregarle tres efectos luminosos adicionales (total cuatro), el cambio se hará mediante un solo pulsador ubicado en RB0.
- Selección de la visualización



Ejercicio adicional 2026-0

- Agregar un pulsador adicional para cambiar la velocidad de lento a rápido.

Fin de la sesión